SUZ CRM — Manual de Explicaciones Técnicas

Guía de defensa y justificación de arquitectura de software para Desarrollador Junior

Proyecto: SUZ (Gestión Comercial Mayorista) **Stack:** React, Redux, Node.js, Express, Sequelize, MySQL **Versión:** Producción y Calidad UI/UX Completa

1. Estandarización y Sincronización de Nombres (Full Stack)

¿QUÉ PROBLEMA HABÍA?

Existía un desacople de nombres entre la pantalla (Frontend), la API (Controlador) y la Base de Datos (Modelo MySQL):

Frontend (Form / Card)	Controlador (Node.js)	Modelo (Sequelize)	Resultado anterior
`contacto`	*no existía*	*no existía*	❌ Se perdía el nombre de contacto
`frecuencia`	`frecuenciaCompra`	`FrecuenciaDecompra`	❌ MySQL guardaba NULL o fallaba
`cantidadKg`	`cantidadKg`	`cantidadDeKg`	❌ Pérdida de kilos facturados
`precioKg`	`precioKg`	`precioPorKg`	❌ Pérdida de precio por kilo

¿QUÉ SE HIZO EN EL CÓDIGO?

Se unificaron los nombres con exactitud en las 3 capas: `contacto`, `frecuencia`, `producto`, `marca`, `cantidadKg`, `precioKg`, `monto` y `deuda`.

> 🕵️ **¿Cómo podés explicarlo si te preguntan?**
> *"Teníamos un error de sincronización de nombres: el frontend enviaba los datos como `frecuencia` y `cantidadKg`, pero en la base de datos se habían bautizado como `FrecuenciaDecompra` y `cantidadDeKg`, y además faltaba registrar el campo `contacto`. Lo que hice fue normalizar los nombres en el modelo y en el controlador para que toda la información viaje directo de la pantalla a MySQL sin perderse en el camino."*

2. Migración Segura en MySQL ({ alter: true } vs { force: true })

¿QUÉ PROBLEMA CRÍTICO HABÍA CON LA BASE DE DATOS?

Había riesgos graves para los datos de la empresa:

1. **Riesgo de pérdida total de datos:** En el script `crear_tablas.js` figuraba `sync({ force: true })`. El parámetro `force: true` ejecuta un `DROP TABLE` destructivo que borra todas las filas existentes de clientes reales.
2. **Tablas desactualizadas en el arranque:** En `Server/src/index.js` solo se autenticaba la conexión con MySQL, pero nunca se sincronizaban las tablas; las columnas nuevas no se creaban en el servidor.
3. **Error de rango numérico en Deuda:** Los campos `monto` y `deuda` estaban configurados como `DECIMAL(8, 2)`, lo que solo permite importes hasta $999.999. Como un cliente real ("La Reina") adeudaba $1.320.000, MySQL rechazaba la actualización con el error: `Out of range value for column 'deuda'`.

¿QUÉ SE HIZO EN EL CÓDIGO?

- Se configuró **`sequelize.sync({ alter: true })`** tanto en `Server/src/index.js` como en `Server/src/crear_tablas.js`.
- Se ampliaron los campos `monto` y `deuda` a **`DECIMAL(12, 2)`** en `Server/src/models/Clientes.js` para permitir facturaciones mayoristas de hasta 10 mil millones de pesos.
- Se ejecutó la migración: se agregaron físicamente las 6 columnas en MySQL y se preservó intacto el cliente existente con su deuda real de $1.320.000.

> 🗣️ **¿Cómo explicarlo con tus palabras si te preguntan?**
> *"Para proteger la información real de los clientes, cambiamos la estrategia de sincronización de Sequelize. NUNCA usamos `force: true` en producción porque borraría todas las tablas. En su lugar, implementamos `alter: true`. Esto le indica a Sequelize que compare nuestros modelos en JavaScript contra las tablas reales de MySQL y ejecute sentencias `ALTER TABLE` para crear únicamente las columnas nuevas sin tocar los registros guardados. Además, ampliamos el tipo de dato a `DECIMAL(12, 2)` para que el sistema soporte facturaciones y deudas mayores a un millón de pesos sin errores de desbordamiento de memoria."*

3. Lógica del Semáforo Comercial (Próxima Compra)

¿QUÉ VALOR COMERCIAL APORTA?

El cliente necesitaba que el sistema no fuera una simple agenda estática, sino una herramienta para que los vendedores sepan a quién llamar cada mañana para reponer mercadería (harina/azúcar).

¿CÓMO SE CALCULA EN EL CÓDIGO?

En `Card.js` se programó la función `calcularCicloComercial()` :

- Toma la `fecha` de la última compra y le suma los días de `frecuencia` (7, 15 o 30 días).
- Calcula la diferencia de días respecto a la fecha actual:
  - 🟢 **En fecha (> 2 días restantes):** El cliente todavía tiene stock suficiente.
  - 🟡 **¡Toca comprar HOY! (0 a 2 días):** Momento exacto de contactarlo por WhatsApp.
  - 🔴 **Vencido (< 0 días):** Pasó su fecha habitual; alerta para retener la cuenta comercial.

> 🗣️ **¿Cómo explicarlo con tus palabras si te preguntan?**
> *"Convertimos la vista en un CRM inteligente: sumamos la fecha de la última compra más la frecuencia habitual del cliente y generamos automáticamente una fecha de próxima compra estimada. Le agregamos un semáforo visual con tres estados para que el vendedor vea de un solo vistazo a quién tiene que reponerle hoy."*

4. Cuenta Corriente y Acción Saldar en Vivo

¿CÓMO FUNCIONA EL CÁLCULO FINANCIERO?

Evitamos que el usuario haga cálculos mentales y restas que provoquen errores:

- **Monto Total:** Se calcula multiplicando `cantidadKg * precioKg` .
- **Monto Pagado:** El dinero que el cliente efectivamente entregó.
- **Deuda:** Se calcula automáticamente como `Monto Total - Monto Pagado` (resaltada en rojo si debe dinero).
- **Botón Saldar:** En un solo clic actualiza la deuda a $0 y equipara el monto pagado al total.

> 🗣️ **¿Cómo explicarlo con tus palabras si te preguntan?**
> *"Separamos con claridad el Monto Total del Monto Pagado. El total se calcula solo multiplicando kilos por precio, y la deuda se actualiza en tiempo real como la diferencia restante. Al hacer clic en Saldar, el sistema limpia la deuda automáticamente."*

5. Control de Acceso y Roles por Área (Login y ProtectedRoute)

¿QUÉ PROBLEMA COMERCIAL Y DE SEGURIDAD RESUELVE?

Originalmente cualquier persona que abría la aplicación podía navegar libremente y modificar clientes de cualquier provincia sin identificarse. En una distribuidora con equipos de venta zonales:

- Un vendedor de **Chaco** no debe interferir con la cartera de clientes de **Corrientes** ni del **Interior de Chaco**.
- El vendedor solo debe ver su área de trabajo asignada para no confundirse de rutas ni alterar saldos ajenos.
- Únicamente la **Administración** debe tener potestad para alternar entre todas las provincias y dar de alta clientes de manera global.

¿CÓMO SE IMPLEMENTÓ EN EL CÓDIGO? (ARQUITECTURA SIMPLE Y LEGIBLE)

Diseñamos una solución liviana sin librerías pesadas, basada en cuatro componentes fundamentales:

1. **Módulo de Autenticación (`src/utils/auth.js`):** Define credenciales por zona y persiste en `localStorage`.
2. **Componente Guardián de Rutas (`src/components/ProtectedRoute.jsx`):** Verifica sesión y redirige automáticamente al login o a la provincia autorizada.
3. **Barra Superior de Sesión (`src/components/HeaderNav/HeaderNav.jsx`):** Badges por área, botón de logout y conmutador exclusivo para el Admin.
4. **Pantalla de Login con Demo Buttons (`src/views/Login/Login.jsx`):** Accesos rápidos con 1 clic para testing y presentación.

> 🕵️ **¿Cómo explicarlo con tus palabras si te preguntan?**
> *"Para organizar el trabajo comercial y proteger la información confidencial entre zonas, implementamos un control de acceso por roles. Creamos un componente envoltorio llamado `ProtectedRoute` en React Router. Cuando alguien intenta acceder a una URL, primero chequeamos si tiene sesión activa en `localStorage`; si no la tiene, va directo a la pantalla de Login. Y si un vendedor intenta meterse por enlace directo a la cartera de otra provincia, el sistema intercepta la petición y lo redirige automáticamente a su zona asignada. El usuario Administrador es el único con permisos globales para auditar todas las áreas y alternar entre provincias desde la barra superior."*

6. Credenciales y Atajos de Demostración

Usuario	Contraseña	Rol / Permiso	Área Asignada	Destino Inicial
admin	`123`	Administrador General	Todas las provincias	`/` (Selector General)
chaco	`123`	Vendedor Chaco	Chaco (Resistencia)	`/Chaco`
corrientes	`123`	Vendedor Corrientes	Corrientes Capital	`/corrientes`
interior	`123`	Vendedor Interior	Interior de Chaco	`/Interior`

7. Seguridad, Variables de Entorno (.env) y Preparación para Producción

¿QUÉ PROBLEMAS EXISTÍAN?

Contraseñas de base de datos en texto plano en `db.js`, URLs fijas `http://localhost:3001` dispersas en 6 archivos del frontend y CORS bloqueado en localhost.

¿CÓMO SE RESOLVIÓ EN EL CÓDIGO?

- **Backend:** Archivo `Server/.env` con claves de MySQL y puerto, leído con `dotenv`.
- **Frontend:** Módulo centralizador `src/utils/config.js` que exporta `API_URL = process.env.REACT_APP_API_URL`.
- **Git:** Archivo `.gitignore` reforzado para no subir secretos y plantillas `.env.example` para colaboradores.

> 🗣️ **¿Cómo explicarlo con tus palabras si te preguntan?**
> *"Por buenas prácticas de seguridad y mantenibilidad, desacoplamos la configuración del código fuente mediante variables de entorno. Nunca se deben dejar credenciales de base de datos ni URLs de red escritas fijas en el código. Implementamos `dotenv` en el servidor para gestionar la conexión a MySQL de manera privada. En el frontend, centralizamos la URL de la API en un único módulo `config.js` con el prefijo `REACT_APP_`. Por último, configuramos `.gitignore` para que jamás suba los archivos `.env` y dejamos plantillas `.env.example` documentadas para el equipo."*

8. Pulido UI/UX: Notificaciones Toast vs Alerts Nativos y Limpieza de Código

¿QUÉ PROBLEMA DE EXPERIENCIA DE USUARIO HABÍA?

Las ventanas emergentes `alert()` del navegador pausaban la ejecución de JavaScript y tenían una estética gris anticuada que rompía el Dark Mode. Además, existían variables sin uso en Redux (`CREAT_CLIENTE`) que producían advertencias.

¿CÓMO SE RESOLVIÓ EN EL CÓDIGO?

1. **Contexto Toast (`src/context/ToastContext.jsx`):** Notificaciones flotantes no bloqueantes con soporte para `success`, `error`, `warning` e `info` con cierre automático a los 3.5 segundos.
2. **Integración en Eventos Clave:** Registro de clientes, saldar deudas y acciones del CRM.
3. **Build 100% Limpio:** Se eliminaron advertencias de ESLint, logrando **0 warnings y 0 errores**.

> 🗣️ **¿Cómo explicarlo con tus palabras si te preguntan?**
> *"Para darle una terminación profesional y moderna al sistema, eliminamos las alertas emergentes `alert()` del navegador que congelaban la pantalla y arruinaban el diseño Dark Mode. Implementamos un sistema propio y liviano de notificaciones Toast usando React Context. Ahora, cada vez que un usuario registra un cliente, salda una cuenta o comete un error de validación, aparece una tarjeta animada y elegante en la esquina superior que informa la acción y desaparece sola a los 3 segundos sin obligar al usuario a hacer clic. Además, limpiamos las advertencias de código en Redux para asegurar un empaquetado de producción 100% limpio."*

9. Clarificación de Dominio Comercial: "Interior" a "Interior de Chaco"

¿QUÉ AMBIGÜEDAD COMERCIAL EXISTÍA?

El término genérico *"Interior"* resultaba ambiguo para los equipos de venta y la administración, ya que en el Nordeste argentino podía aludir al interior de Corrientes o de cualquier otra provincia. En la logística real de la empresa distribuidora mayorista, las zonas comerciales están definidas como:

- **Chaco (Gran Resistencia):** Clientes de la capital provincial y ejido urbano cercano.
- **Corrientes:** Clientes de Corrientes Capital.
- **Interior de Chaco:** Localidades y pueblos chaqueños (Presidencia Roque Sáenz Peña, Villa Ángela, Castelli, Charata, etc.).

¿CÓMO SE AJUSTÓ EN LA INTERFAZ Y EN EL CÓDIGO?

- **Portada Landing (`Landing.jsx`):** El botón de acceso se renombró a `INTERIOR DE CHACO` y `CHACO (RESISTENCIA)` para máxima claridad visual.
- **Encabezado de la Vista (`Interior.jsx`):** Se actualizó el título principal a `Clientes en Interior de Chaco`.
- **Barra Superior y Perfil (`HeaderNav.jsx` y `auth.js`):** La barra de navegación para el Administrador indica `Interior de Chaco` y el vendedor asignado muestra el distintivo `📍 Interior de Chaco`.
- **Botonera de Login (`Login.jsx`):** Se actualizó el botón de acceso rápido de la demo a `📍 Interior de Chaco`.
- **Formulario Comercial (`Form.jsx`):** Se reemplazó el campo de texto libre por un `<select>` desplegable con las tres zonas exactas para imposibilitar errores de tipeo.
- **Compatibilidad en MySQL:** Se preservó el identificador interno `"interior"` en la base de datos para no alterar las consultas históricas ni quebrar registros existentes.

> 🗣️ **¿Cómo explicarlo con tus palabras si te preguntan?**
> *"El término 'Interior' era demasiado genérico y generaba confusión en la operativa comercial, porque no aclaraba a qué provincia pertenecía. Lo estandarizamos en toda la interfaz como 'Interior de Chaco' para diferenciarlo con total precisión de Resistencia Capital y de Corrientes. En el formulario convertimos el ingreso de zona en un selector desplegable con las tres opciones claras, evitando que los usuarios cometan errores ortográficos, y mantuvimos el valor 'interior' a nivel base de datos para preservar la compatibilidad con todas las consultas previas."*